Singleton vs Scoped vs Transient in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

Singleton vs Scoped vs Transient in ASP.NET Core Web API

In **ASP.NET Core**, Dependency Injection (DI) is built in and helps manage **object lifetimes** and **dependencies** efficiently. When registering services in the DI container, you choose how long the service should live using one of these lifetimes:

- Singleton
- Scoped
- Transient

Each lifetime determines:

- **How many instances of the service will be created**
- **When they are created**
- **How long are they reused**

Singleton Lifetime

- **How many instances are created?** Only **one instance** is created throughout the entire lifetime of the application.
- **When is it created?** Created **when first requested** (lazy initialization) or immediately at startup if you use AddSingleton with a pre-built instance.
- **How long is it reused?** The **same instance** is reused **for every request and every user** until the application stops.

Scoped Lifetime

- **How many instances are created?** One instance **per request (scope)** is created.
- **When is it created?** Created **once per HTTP request**, when the service is first needed during that request.
- **How long is it reused?** It is reused **throughout the same request**, but **discarded** at the end of the request.

Transient Lifetime

- **How many instances are created?** A **new instance** is created **every time** it's requested.
- **When is it created?** Created **each time** a class or controller asks for it, even within the same request.

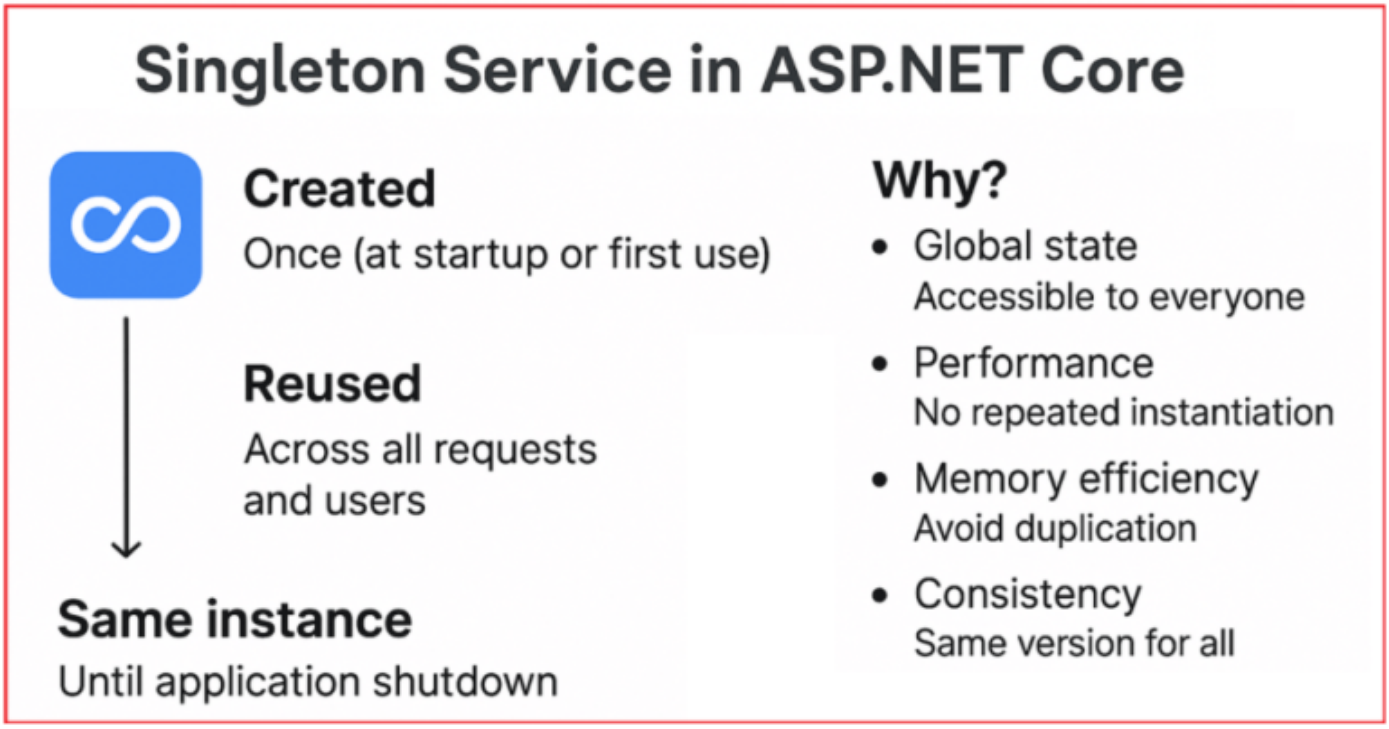
- **How long is it reused?** It is **never reused**; each injection gets its own separate instance.

Singleton Service in ASP.NET Core

A **Singleton Service** is the simplest and most powerful way to share one instance across the **entire Application Lifetime**. That means:

- It is **created only once** (at application startup or on the first time it's requested).
- It is then **reused across all HTTP requests, all controllers, and all users**.
- The same object instance lives until the application shuts down.

For a better understanding, please refer to the following diagram.



Why Singleton Service in ASP.NET Core Web API?

Singleton services are best suited when you want **a single shared resource** that does not depend on request-specific data. Some key benefits:

1. **Global State** → If you want to keep something in memory and accessible to everyone (such as a tax configuration or a cache), a Singleton works perfectly.
2. **Performance** → Since the service is created only once, you avoid the cost of creating a new instance repeatedly.
3. **Memory Efficiency** → Objects that are expensive to create (e.g., database connection pools, API clients, machine learning models, etc.) don't need to be duplicated for every request.
4. **Consistency** → All requests see the same version of the service.

How ASP.NET Core Framework Manages Singleton:

The ASP.NET Core Dependency Injection (DI) container manages the **lifecycle automatically**:

- **Creation**: When the app starts, or when the service is first requested, ASP.NET Core creates the instance.
- **Reuse**: Every controller, service, or component that depends on it will get the same object reference.
- **Disposal**: When the application shuts down, ASP.NET Core disposes the Singleton (releases resources, calls Dispose() if implemented).

You don't need to manage it manually; DI does the work.

Use Case of Singleton Service in ASP.NET Core:

Some common real-world scenarios where Singleton makes sense:

- **Caching** → Store frequently used data (e.g., product categories, configuration, lookup tables) in memory for fast access.
- **Configuration Providers** → A central service that loads configuration settings (e.g., appsettings.json, environment variables) and provides them to other services.
- **Logging** → A logging service can be shared across the entire app (all requests log to the same logger).
- **External API Clients** → If your app needs to call external APIs (e.g., payment gateway, weather API), you don't want to create a new HTTP client every time — instead, use IHttpClientFactory or a Singleton service.

Real-World Analogy:

Think of a **TV Remote** at home:

- Only one remote exists for the TV.
- Everyone (parents, kids, guests) uses the same remote to control the TV.
- You don't need a new remote for every person or every button press.
- Everyone reuses the same instance (remote) until it breaks or the batteries die.

In short: A Singleton service is **ideal for global, shared, and reusable logic** that does not depend on per-request or per-user data.

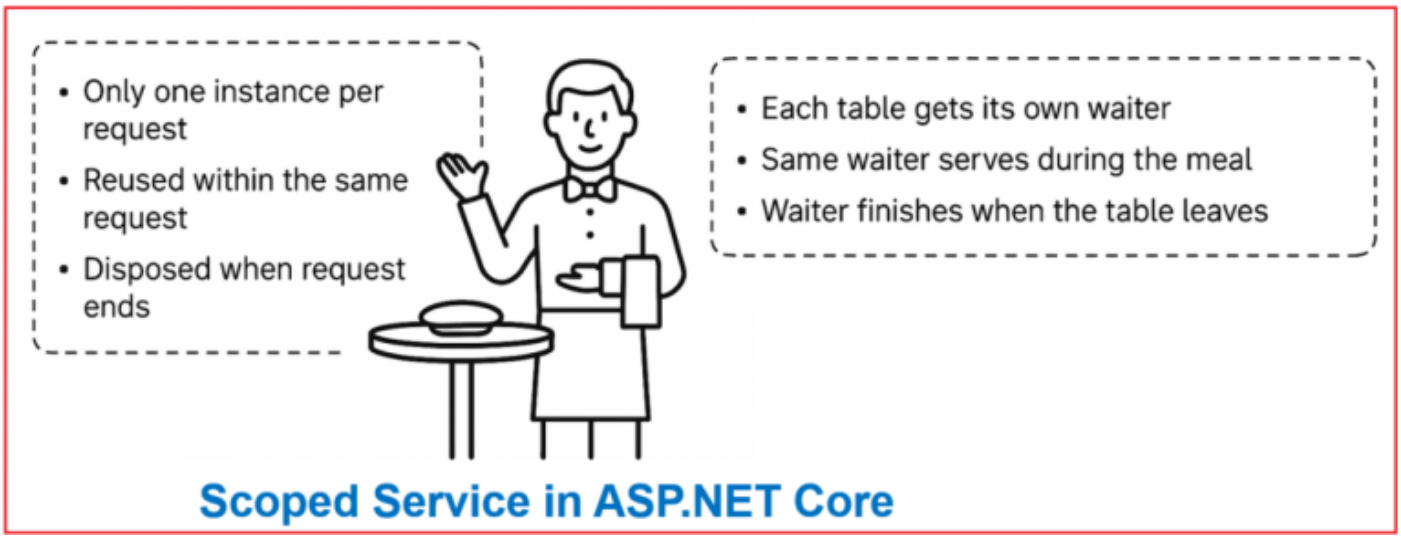
Scoped Service in ASP.NET Core

A **Scoped service** is created **once per HTTP request**.

- Within a single request, the **same instance is reused** everywhere it's injected.
- But as soon as a new HTTP request starts, ASP.NET Core creates a **new instance** for that request.

- Once the request ends, the instance is **disposed** (cleaned up by the DI container).

So, it lives only as long as the HTTP request scope. For a better understanding, please refer to the following diagram.



Why Scoped Service in ASP.NET Core Web API?

Scoped services are useful when you need to **maintain state per request**, but you don't want that state to leak across different users or requests.

Examples:

- A **shopping cart** that exists only while a request is being processed.
- A **UserContext service** that holds info about the currently logged-in user.
- A **Unit of Work** service where one request = one transaction.

Scoped ensures **isolation** → each request gets its own copy, but consistency inside that request.

How ASP.NET Core Framework Manages Scoped:

1. When a new HTTP request comes in → a **Scoped service instance is created**.
2. If multiple controllers, services, or components require the same instance within the same request, they all receive the **same instance**.
3. When the request ends → ASP.NET Core **disposes** the instance (releases resources).

This ensures the state is consistent within the request, but is safely discarded after.

Real-World Analogy:

Think of a **waiter in a restaurant**:

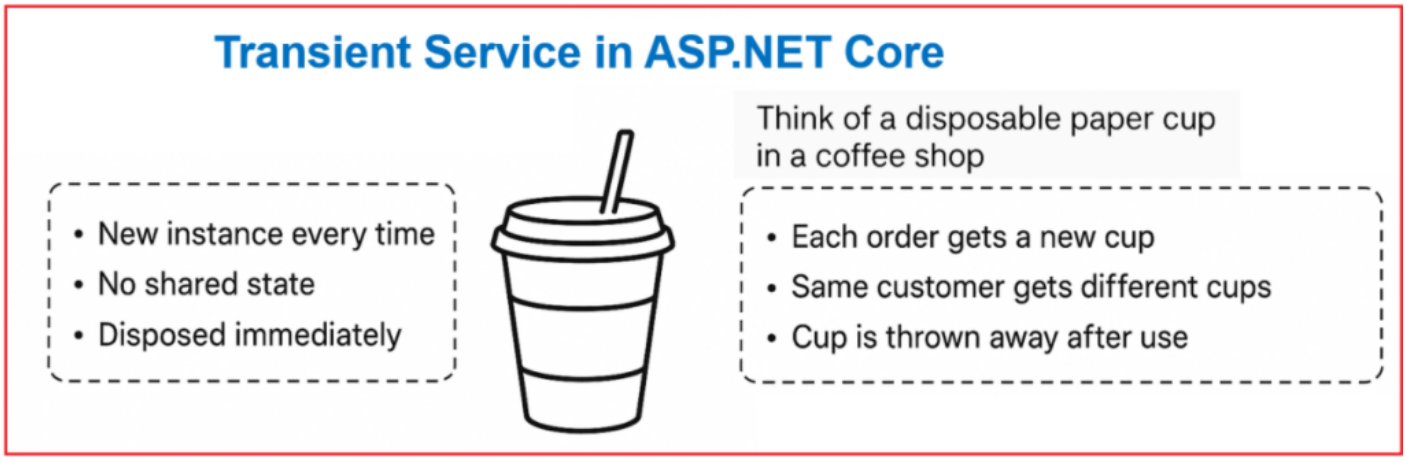
- Each table (like an HTTP request) gets its **own waiter**.
- During the meal, the waiter consistently serves you (the same instance is reused across that request).
- When the table leaves, the waiter's job for that table is finished (disposed).
- The next table receives a **new waiter** (a new Scoped instance).

Transient Service in ASP.NET Core

A **Transient service** is the **shortest-lived service** in ASP.NET Core's Dependency Injection system.

- A **new instance is created every time** it is requested from the DI container.
- Even within the same request, if you inject it twice, you will get **two completely separate objects**.
- As a result, the service **doesn't hold any shared state** across or within requests.
- Once used, it's disposed (garbage collected) quickly, no reuse at all.

This makes transient services **ideal for lightweight, stateless tasks** that don't rely on user data, shared context, or long-lived memory. For a better understanding, please refer to the following diagram.



Why Transient Service in ASP.NET Core Web API?

Use Transient when:

- Your service is **lightweight** (not expensive to create).
- Your service does **not hold state** (stateless).
- You want to **guarantee isolation** (no shared data).

How ASP.NET Core Framework Manages Transient:

1. When a Transient service is requested (via constructor injection, GetService(), etc.), ASP.NET Core **creates a brand new instance**.
2. If injected multiple times in the same controller/service, each injection results in a **different instance**.
3. Once it goes out of scope, it's disposed immediately (if disposable).

When to Use Transient Service in ASP.NET Core:

Best for services that are **stateless and lightweight**:

- **Calculation helpers** → e.g., discount calculation, price formatting, currency conversion.
- **Utility classes** → string formatters, mappers, validators.
- **Random generators / ID generators** → if you specifically want a different value each time.
- **Stateless API wrappers** → if they don't need to cache or persist anything.

Real-World Analogy:

Think of a **disposable paper cup in a coffee shop**:

- Each order gets a **new cup**.
- Even if the same customer orders twice, they get **two different cups**.
- Cups are **not reused or shared** between customers.
- After use, the cup is thrown away (disposed).

That's exactly how Transient services behave.

Real-Time Example: Shopping Cart in ASP.NET Core Web API

Imagine an e-commerce system.

- The **CartService** stores items added by the user.
- The **DiscountService** provides dynamic discounts.
- The **AppConfigService** provides global configuration (like tax rate, delivery fee).

Now, we will implement these with **different lifetimes**:

Step 1: Create a New ASP.NET Core Web API Project

First, create a new ASP.NET Core Web API Project, name it ShoppingCartAPI.

Step 2: Define Models

Create a new folder, Models, at the project root directory. Then, add the following classes inside the Models folder.

CartItem.cs

Represents an item added to the cart, including ProductId, Name, Price, and Quantity. Create a class file named **CartItem.cs** within the **Models** folder, and then copy-paste the following code.

```
namespace ShoppingCartAPI.Models
{
    public class CartItem
    {
        public int ProductId { get; set; }
        public string ProductName { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public int Quantity { get; set; }
    }
}
```

CartSummary.cs

Represents the final computed values of a cart: Subtotal, Discount, Tax, Delivery Fee, and Final Total. Create a class file named **CartSummary.cs** within the **Models** folder, and then copy-paste the following code.

```
namespace ShoppingCartAPI.Models
{
    public class CartSummary
    {
        public decimal SubTotal { get; set; }
        public decimal Discount { get; set; }
        public decimal Tax { get; set; }
        public decimal DeliveryFee { get; set; }
        public decimal Total { get; set; }
    }
}
```

Step 3: Define Services

Create a new folder **Services** at the project root directory. Then, add the following classes and interfaces inside the Services folder.

Singleton Service → AppConfigService

Create an interface named **IAppConfigService.cs** within the **Services** folder, and then copy-paste the following code.

```
namespace ShoppingCartAPI.Services
{
    public interface IAppConfigService
    {
        decimal GetTaxRate();
        decimal GetDeliveryFee(decimal orderAmount);
    }
```

```
}
```

AppConfigService

This class provides application-wide configuration data, such as the **GST tax rate** and **delivery fee,** based on the cart amount. These values don't change per request or user. Create a class file named **AppConfigService.cs** within the **Services** folder, and then copy-paste the following code.

```
namespace ShoppingCartAPI.Services
{
    public class AppConfigService : IAppConfigService
    {
        private readonly decimal _taxRate = 0.18m; // GST rate = 18%

        public AppConfigService(ILogger<AppConfigService> logger)
        {
            // Log only once since Singleton → single instance shared across app
            logger.LogInformation("AppConfigService (Singleton) instance
created.");
        }

        // Get current tax rate
        public decimal GetTaxRate()
        {
            return _taxRate;
        }

        // Calculate delivery fee based on order amount
        public decimal GetDeliveryFee(decimal orderAmount)
        {
            if (orderAmount < 500)
                return 50;  // Flat ₹50 for small orders
            else if (orderAmount >= 500 && orderAmount <= 2000)
                return 30;  // Reduced fee for mid-sized orders
            else
                return 0;   // Free delivery for big orders
        }
    }
}
```

Why Singleton?

- It holds static, read-mostly configuration.
- It's shared across all users and requests.
- Saves memory and avoids unnecessary object creation.

Scoped Service → CartService

Stores the **shopping cart** for a single user request. Create an interface named **ICartService.cs** within the **Services** folder, and then copy-paste the following code.

```
using ShoppingCartAPI.Models;
namespace ShoppingCartAPI.Services
{
    public interface ICartService
    {
        void AddItem(CartItem item);
        List<CartItem> GetItems();
        void ClearCart();
    }
}
```

CartService

This service manages the **user's shopping cart** for the current HTTP request. It fetches the cart from IMemoryCache, adds/updates items, and clears the cart as needed. Create a class file named **CartService.cs** within the **Services** folder, and then copy-paste the following code.

```
using Microsoft.Extensions.Caching.Memory;
using ShoppingCartAPI.Models;

namespace ShoppingCartAPI.Services
{
    public class CartService : ICartService
    {
        private readonly IMemoryCache _cache;  // Used to store carts in memory
        private readonly string _userId;       // Unique identifier for the user
(from request header)

        // Constructor: runs once per HTTP request (because CartService is
registered as Scoped)
        public CartService(IMemoryCache cache, IHttpContextAccessor
httpContextAccessor, ILogger<CartService> logger)
        {
            // Initialize Memory Cache
            _cache = cache;

            // Get UserId from request header if available, otherwise fallback to
"guest"
            _userId =
httpContextAccessor.HttpContext?.Request.Headers["UserId"].FirstOrDefault() ??
"guest";
```

```
            // Log only when a new instance of CartService is created (helps
visualize Scoped lifetime)
            logger.LogInformation("CartService (Scoped) instance created for user
{UserId}", _userId);
        }

        // Add or update a cart item for the current user
        public void AddItem(CartItem item)
        {
            // Try to get cart from memory; if not found, create a new one with
30-minute sliding expiration
            var cart = _cache.GetOrCreate(_userId, entry =>
            {
                entry.SlidingExpiration = TimeSpan.FromMinutes(30); // Reset
expiration timer when accessed
                return new List<CartItem>(); // Start with empty cart for this
user
            })!; // The "!" tells compiler: "this will never be null"

            // Check if product already exists in the user's cart
            var existing = cart.FirstOrDefault(x => x.ProductId ==
item.ProductId);

            if (existing != null)
                existing.Quantity += item.Quantity; // If already exists,
increase quantity
            else
                cart.Add(item); // Otherwise, add as a new product

            // Save the updated cart back into memory cache for this user
            _cache.Set(_userId, cart);
        }

        // Get all items from the user's cart
        public List<CartItem> GetItems()
        {
            // Try to retrieve cart from cache
            if (_cache.TryGetValue(_userId, out List<CartItem>? cart))
            {
                // If found, return cart (safe-checked with ?? to avoid null
warning)
                return cart ?? new List<CartItem>();
            }

            // If cart not found for this user, return empty list
            return new List<CartItem>();
```

```
        }

        // Clear all items in the user's cart
        public void ClearCart()
        {
            // Simply remove the entry from memory cache
            _cache.Remove(_userId);
        }
    }
}
```

Why Scoped?

- A new instance is created per request.
- It ensures **request isolation**, but the **user cart persists** across requests via IMemoryCache (singleton).
- Allows logging and handling per-user state safely within a request.

Transient Service → DiscountService

Provides a **tier-based discount logic** (5%, 10%, 15%) depending on the subtotal amount. Create an interface named **IDiscountService.cs** within the **Services** folder, and then copy-paste the following code.

```
namespace ShoppingCartAPI.Services
{
    public interface IDiscountService
    {
        decimal CalculateDiscount(decimal amount);
    }
}
```

DiscountService

Calculates discounts based on order amount (5%, 10%, 15%). Each injection creates a new instance so that you can see multiple calculations in the same request. Create a class file named **DiscountService.cs** within the **Services** folder, and then copy-paste the following code.

```
namespace ShoppingCartAPI.Services
{
    public class DiscountService : IDiscountService
    {
        public DiscountService(ILogger<DiscountService> logger)
        {
            // Log whenever a new DiscountService instance is created
            // (helps visualize Transient lifetime → multiple per request
possible)
```

```
            logger.LogInformation("DiscountService (Transient) instance
created.");
        }

        public decimal CalculateDiscount(decimal amount)
        {
            // Tier-based discount calculation
            decimal discountPercent = 0;

            if (amount >= 5000 && amount <= 20000)
                discountPercent = 5;   // 5% discount for mid-level orders
            else if (amount > 20000 && amount <= 50000)
                discountPercent = 10;  // 10% discount for higher orders
            else if (amount > 50000)
                discountPercent = 15;  // 15% discount for premium orders

            // Calculate discount amount
            decimal discount = amount * discountPercent / 100;

            return discount;
        }
    }
}
```

Why Transient?

- It is **stateless** and lightweight.
- We want **a new instance every time** (even within the same request), especially to show how different injections behave independently.
- Suitable for one-off calculations.

Scoped Service → ICartSummaryService

Create an interface named **ICartSummaryService.cs** within the **Services** folder, and then copy-paste the following code.

```
using ShoppingCartAPI.Models;
namespace ShoppingCartAPI.Services
{
    public interface ICartSummaryService
    {
        CartSummary GenerateSummary();
    }
}
```

CartSummaryService

Generates the **final cart summary** (Subtotal, Discount, Tax, Delivery Fee, and Total). Depends on:

- CartService (Scoped → per-request cart data),
- DiscountService (Transient → fresh calculations),
- AppConfigService (Singleton → global tax/delivery rules).

Create a class file named **CartSummaryService.cs** within the **Services** folder, and then copy-paste the following code.

```
using ShoppingCartAPI.Models;
namespace ShoppingCartAPI.Services
{
    // Service responsible for generating the Cart Summary
    public class CartSummaryService : ICartSummaryService
    {
        // Dependencies injected via constructor
        private readonly ICartService _cartService;       // To fetch items
already added to the cart
        private readonly IDiscountService _discount1;     // First transient
discount service
        private readonly IDiscountService _discount2;     // Second transient
discount service (for demo showing new instance)
        private readonly IAppConfigService _config;       // For global
configuration (tax rate, delivery fee)

        // Constructor injection - services are provided by DI container
        public CartSummaryService(
            ICartService cartService,
            IDiscountService discount1,
            IDiscountService discount2,
            IAppConfigService config)
        {
            _cartService = cartService;
            _discount1 = discount1;
            _discount2 = discount2;
            _config = config;
        }

        // Main method to calculate and return cart summary
        public CartSummary GenerateSummary()
        {
            // Fetch all cart items from the cart service
            var items = _cartService.GetItems();
```

```
            // Calculate subtotal = sum of price * quantity for all items
            decimal subTotal = items.Sum(i => i.Price * i.Quantity);

            // Apply discounts using two transient instances
            // (each transient service will likely give different results,
            // to demonstrate lifetime differences)
            decimal discount1 = _discount1.CalculateDiscount(subTotal);
            decimal discount2 = _discount2.CalculateDiscount(subTotal);

            // Calculate tax using the Singleton AppConfigService
            decimal tax = subTotal * _config.GetTaxRate();

            // Calculate delivery fee using dynamic business rules
            decimal delivery = _config.GetDeliveryFee(subTotal);

            // Build and return the summary object
            return new CartSummary
            {
                SubTotal = subTotal,                                // Original
total before tax/discounts
                Discount = (discount1 + discount2) / 2,             // Average of
both discounts (demo purpose)
                Tax = tax,                                          // Calculated
tax
                DeliveryFee = delivery,                             // Delivery fee
based on subtotal
                Total = subTotal - ((discount1 + discount2) / 2)  // Net total =
subtotal - discount + tax + delivery
                        + tax
                        + delivery
            };
        }
    }
}
```

Why Scoped?

- It depends on the scoped CartService, transient DiscountService, and singleton AppConfigService.
- Scoped ensures a **fresh summary per request**, yet internally respects the correct lifetimes of its dependencies.

Step 4: Register Services in Program.cs

Now, we need to register the services with a proper lifetime. So, please update the **Program.cs** class file as follows:

```
using ShoppingCartAPI.Services;
```

```
namespace ShoppingCartAPI
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers();
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            // Service Register lifetimes
            // Register AppConfigService as Singleton
            // One instance is created and shared across the entire application
lifetime
            //    - Good for global config like tax rate, delivery fee
            //    - Created once, reused everywhere
            builder.Services.AddSingleton<IAppConfigService, AppConfigService>();

            // Register CartService as Scoped
            // One instance per HTTP request
            //    - Each request gets its own CartService instance
            //    - Items stored in CartService are isolated to that request
            //    - Demonstrates per-request lifetime
            builder.Services.AddScoped<ICartService, CartService>();

            // Register DiscountService as Transient
            // A new instance is created every time it is requested
            //    - Even within the same request, multiple injections create
different objects
            //    - Great for lightweight, stateless operations like discount
calculations
            builder.Services.AddTransient<IDiscountService, DiscountService>();

            // Register CartSummaryService as Scoped
            // New instance per HTTP request
            //    - Depends on CartService, DiscountService, AppConfigService
            //    - Calculates cart totals (subtotal, discount, tax, delivery,
final total)
            //    - Scoped makes sense because summary is tied to the current
cart/request
            builder.Services.AddScoped<ICartSummaryService, CartSummaryService>
();

            // Register HttpContextAccessor
```

```
            // Provides access to HttpContext (e.g., reading headers like
"UserId")
            //    - Needed for CartService to know which user's cart to manage
            builder.Services.AddHttpContextAccessor();

            // Register In-Memory Cache
            // Provides IMemoryCache (Singleton under the hood)
            //    - Used by CartService to persist cart data across requests for
a given user
            //    - Supports expiration policies (e.g., cart expires after 30
minutes of inactivity)
            builder.Services.AddMemoryCache();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

Step 5: Creating Cart Controller

Handles the HTTP endpoints:

- POST /add to add an item to the cart
- GET /items to view cart contents
- GET /summary to calculate totals
- DELETE /clear to empty the cart

It demonstrates how services with different lifetimes work together to build a real-time user experience. So, create an Empty API Controller named **CartController** within the **Controllers** folder, and then copy-paste the following code.

```
using Microsoft.AspNetCore.Mvc;
using ShoppingCartAPI.Models;
using ShoppingCartAPI.Services;

namespace ShoppingCartAPI.Controllers
{
    // Marks this as a Web API controller
    // [ApiController] enables automatic model validation and better request
handling
    [ApiController]

    // Base route → all endpoints will be under "api/cart"
    [Route("api/[controller]")]
    public class CartController : ControllerBase
    {
        // Injected dependency for cart operations (Scoped service)
        private readonly ICartService _cartService;

        // Constructor Injection
        // ASP.NET Core automatically resolves ICartService from the DI container
        public CartController(ICartService cartService)
        {
            _cartService = cartService;
        }

        // POST: api/cart/add
        // Adds an item to the user's cart
        [HttpPost("add")]
        public IActionResult AddItem(CartItem item)
        {
            // Call cart service to add item to the in-memory cache
            _cartService.AddItem(item);

            // Return success response with a message
            return Ok(new { Message = $"{item.ProductName} added to cart." });
        }

        // GET: api/cart/items
        // Returns all items currently in the user's cart
        [HttpGet("items")]
        public IActionResult GetItems()
        {
            // Fetch items from cart service
            return Ok(_cartService.GetItems());
        }

        // GET: api/cart/summary
```

```
        // Returns the calculated cart summary (subtotal, discounts, tax,
delivery fee, total)
        [HttpGet("summary")]
        public IActionResult GetSummary([FromServices] ICartSummaryService
summaryService)
        {
            // Here, CartSummaryService is injected per-request (Scoped)
            // It internally uses:
            //   - ICartService (Scoped, manages cart data)
            //   - IDiscountService (Transient, two different instances for
discount calculations)
            //   - IAppConfigService (Singleton, global tax & delivery fee rules)
            var summary = summaryService.GenerateSummary();

            // Return summary object as JSON response
            return Ok(summary);
        }

        // DELETE: api/cart/clear
        // Clears all items from the user's cart
        [HttpDelete("clear")]
        public IActionResult ClearCart()
        {
            // Clear user's cart from cache
            _cartService.ClearCart();

            // Return success response
            return Ok(new { Message = "Cart cleared." });
        }
    }
}
```

Step 6: Test API in Postman / Swagger

Request 1: Add Laptop

POST /api/cart/add

```
{
  "productId": 1,
  "productName": "Laptop",
  "price": 60000,
  "quantity": 1
}
```

Here:

- A **Scoped CartService** instance is created for this request.
- The Laptop item is added to the cart (stored in **IMemoryCache**, keyed by UserId).
- **Log Output**:
    - CartService (Scoped) instance created for user user123
    - If this is the very first request, you'll also see: AppConfigService (Singleton) instance created.

Request 2: Add Mobile

POST /api/cart/add

```
{
  "productId": 2,
  "productName": "Mobile",
  "price": 10000,
  "quantity": 2
}
```

Here,

- A **new Scoped CartService** instance is created for this request (constructor log shows again).
- However, since cart data is stored in **IMemoryCache**, the Laptop from Request 1 is still there.
- After this request, the cart now contains **Laptop + Mobile**.
- **Log Output**:
    - Another CartService (Scoped) instance created for user user123.

This demonstrates how **Scoped creates a new service instance for each request**, but **cart persistence is achieved via Singleton IMemoryCache**.

View Items

GET /api/cart/items

Here,

- Creates another **Scoped CartService** instance.
- Retrieves cart items from **IMemoryCache**.
- You'll see **both Laptop + Mobile**, because cache persists data beyond request lifetime.
- **Log Output**:
    - CartService (Scoped) instance created for user user123.

Get Cart Summary

GET /api/cart/summary

Here,

- Creates a **new Scoped CartService** instance.
- Fetches all cart items (Laptop + Mobile).
- Two **Transient DiscountService** instances are created:
    - _discount1
    - _discount2
- Each calculates a discount **based on the subtotal** (tiered: 5%, 10%, 15%).
- **Singleton AppConfigService** is reused (no new log).
    - It now calculates the **delivery fee dynamically**:
        - Subtotal < 500 → ₹50
        - Subtotal 500–2000 → ₹30
        - Subtotal > 2000 → Free delivery (₹0)

Log Output:

- CartService (Scoped) instance created for user user123
- DiscountService (Transient) instance created.
- DiscountService (Transient) instance created.

Best Practices

- Choose **Singleton** for shared, thread-safe, read-mostly resources you want to reuse.
- Choose **Scoped** for anything that logically lives for a single HTTP request.
- Choose **Transient** for small, stateless helpers you don't mind recreating often.

Understanding these lifetimes is **crucial** for building efficient and bug-free ASP.NET Core Web APIs. Improper use can cause:

- **Memory leaks** (e.g., injecting scoped into a singleton),
- **Unwanted shared states**, or
- **Performance issues**.

Always choose the **right lifetime** based on the **data scope**, **state requirements**, and **performance considerations**.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.